

# Módulo 3 — Variáveis, tipos e operadores

Jornada DEV START · Aula 3 · 15/07 (quarta) · Programa START — TOTVS Paulista  
Material entregue ao final da Aula 3. Guarde para consultar sempre que precisar.



## Onde você está na jornada

```
1 → 2 → [3] → 4 → 5 → 6 | 7 → 8 → 9 → 10
      ▲ Fundamentos (Harbour) | Protheus (ADVPL)
      você está aqui
```

Na Aula 2 você aprendeu a **pensar** a solução no papel. Hoje começamos a escrever **código de verdade**: como o computador guarda informações nos “escaninhos” (as **variáveis**), quais **tipos de dados** existem em Harbour/ADVPL e como manipulá-los com **operadores**. Sai o pseudocódigo, entra o código. 💾



## Roteiro da aula (2h)

| Bloco            | Tempo   | O quê                                                                        |
|------------------|---------|------------------------------------------------------------------------------|
| Retomada         | ~10 min | Revisão do Módulo 2 (algoritmos/fluxogramas) + correção de exercício-chave   |
| Conteúdo         | ~60 min | Variáveis, tipos de dados, escopos, operadores, conversão e entrada de dados |
| Mão na massa     | ~40 min | Construir uma calculadora simples ao vivo                                    |
| Q&A / networking | ~10 min | Dúvidas abertas e integração da turma                                        |

## 🎯 Objetivos do módulo

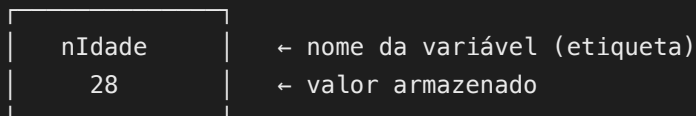
Ao final desta aula você será capaz de: - Entender o conceito de **variável** e como a memória funciona - Conhecer todos os **tipos de dados** do Harbour/ADVPL - Declarar variáveis corretamente ( `LOCAL` , `PRIVATE` , `PUBLIC` , `STATIC` ) - Usar operadores **aritméticos**, **relacionais**, **lógicos** e de **string** - Usar funções matemáticas e de **conversão** - Escrever programas que **leem dados** do usuário e exibem resultados

### 3.1 Variáveis e memória

#### O que é uma variável?

Lembra dos “escaninhos” do computador simplificado (Módulo 2)? Na programação, chamamos esses escaninhos de **variáveis**.

Uma variável é um **espaço nomeado na memória** onde guardamos um valor. Pense em uma **caixa com uma etiqueta**:



 Variável como caixa etiquetada na memória

#### Identificadores

O nome que damos a uma variável é o **identificador**. Regras em Harbour/ADVPL:

- Pode conter letras, números e sublinhado ( `_` )
- Deve **começar** com uma letra ou sublinhado
- Não pode ser uma palavra reservada ( `IF` , `WHILE` , `FUNCTION` , etc.)
- **Não** é case-sensitive ( `nIdade` = `NIDADE` = `nidade` )
- Máximo recomendado: até **10 caracteres** no ADVPL/Protheus (por compatibilidade)

#### Convenção de nomes (Notação Húngara)

No mundo Harbour/ADVPL existe uma convenção importantíssima — o **prefixo de tipo**. A primeira letra do nome indica o **tipo** do dado:

| Prefixo | Tipo               | Exemplo            |
|---------|--------------------|--------------------|
| c       | Caractere (string) | cNome , cCodigo    |
| n       | Numérico           | nIdade , nValor    |
| l       | Lógico (booleano)  | lAtivo , lOk       |
| d       | Data               | dNasc , dVencto    |
| a       | Array (vetor)      | aItens , aClientes |
| o       | Objeto             | oConexao , oErro   |
| b       | Bloco de código    | bFiltro , bCalc    |

Seguir essa convenção facilita **muito** a leitura do código — batendo o olho no nome, você já sabe o tipo. É padrão em todo código Protheus profissional.

## 3.2 Tipos de dados

Harbour/ADVPL possui os seguintes tipos primitivos.

### Caractere (string)

Representa texto. Deve ser delimitado por aspas duplas ou simples.

```
LOCAL cNome    := "João da Silva"
LOCAL cCidade := 'São Paulo'
LOCAL cCodigo := "001"           // "001" é texto, não número!
```

Funções úteis com strings:

```
QOut(Len("Harbour"))           // 7      - tamanho da string
QOut(Upper("hello"))           // HELLO  - maiúsculas
QOut(Lower("HELLO"))           // hello - minúsculas
QOut(Left("Harbour", 3))       // Har   - primeiros 3 caracteres
QOut(Right("Harbour", 4))      // bour  - últimos 4 caracteres
QOut(SubStr("Harbour", 2, 3))  // arb   - a partir da posição 2, 3 chars
QOut(Trim(" texto "))         // " texto" - remove espaços à direita
QOut(AllTrim(" texto "))      // "texto"  - remove espaços dos dois lados
```

### Numérico

Representa números inteiros ou decimais. **Não** usa aspas.

```
LOCAL nIdade      := 28
LOCAL nSalario    := 3500.50
LOCAL nPi          := 3.14159
LOCAL nNegativo   := -100
```

Funções matemáticas:

```
QOut(Abs(-15))      // 15    - valor absoluto
QOut(Int(3.9))       // 3     - parte inteira (trunca)
QOut(Round(3.456, 2)) // 3.46 - arredonda para 2 decimais
QOut(Sqrt(16))       // 4     - raiz quadrada
QOut(Mod(10, 3))     // 1     - resto da divisão
QOut(Max(5, 12))     // 12    - maior valor
QOut(Min(5, 12))     // 5     - menor valor
```

## Lógico (booleano)

Representa verdadeiro ou falso. Em Harbour/ADVPL usamos `.T.` e `.F.`.

```
LOCAL lAtivo      := .T.      // verdadeiro
LOCAL lBloqueio   := .F.      // falso
LOCAL lMaior      := (18 > 10) // .T.
```

⚠ **Atenção:** os pontos ao redor do `.T.` e `.F.` são **obrigatórios**!

## Data

Representa uma data. Criada com `Date()` (hoje) ou `CtoD()` (a partir de texto).

```
LOCAL dHoje      := Date()      // data de hoje
LOCAL dNasc      := CtoD("15/03/1990") // string para data
LOCAL dVencto    := dHoje + 30   // daqui a 30 dias

QOut(DToC(dHoje)) // converte data para string (exibição)
QOut(Year(dHoje))  // ano
QOut(Month(dHoje)) // mês
QOut(Day(dHoje))   // dia
```

💡 Repare: `dHoje + 30` **soma 30 dias** à data. Datas em Harbour entendem aritmética!

## NIL

Representa **ausência de valor**. Uma variável declarada sem valor atribuído é **NIL**.

```
LOCAL xVar
QOut(xVar == NIL)  // .T.
```

## 3.3 Declaração de variáveis (escopos)

Em Harbour/ADVPL existem quatro formas de declarar variáveis, com **escopos** (visibilidade) diferentes.

### LOCAL

Visível apenas dentro da função onde foi declarada. **A mais usada e recomendada.**

```
FUNCTION MinhaFuncao()
    LOCAL cNome := "Maria"
    LOCAL nIdade
    nIdade := 25
    // cNome e nIdade só existem aqui dentro
    RETURN NIL
```

### PRIVATE

Visível na função atual **e em todas as funções chamadas** a partir dela.

```
FUNCTION Principal()
    PRIVATE cFilial := "01"
    OutraFuncao()      // cFilial é visível aqui dentro também
    RETURN NIL

FUNCTION OutraFuncao()
    QOut(cFilial)      // funciona! vê a variável PRIVATE da chamadora
    RETURN NIL
```

No Protheus, variáveis como **cCadastro** e **aRotina** são **PRIVATE** por convenção.

### PUBLIC

Visível em **todo** o programa (variável global). Deve ser **evitada**.

```
PUBLIC gNomeEmpresa := "ACME Corp"
// Visível em qualquer função do programa
```

## STATIC

Visível apenas na função, mas **mantém o valor entre chamadas**.

```
FUNCTION Contador()
    STATIC nVezes := 0
    nVezes++
    QOut("Chamei " + Str(nVezes) + " vez(es)")
RETURN NIL

// Chamadas:
// Contador() → "Chamei 1 vez(es)"
// Contador() → "Chamei 2 vez(es)"
// Contador() → "Chamei 3 vez(es)"
```

## Resumo dos escopos

| Declaração | Visível onde?                         | Quando usar?                 |
|------------|---------------------------------------|------------------------------|
| LOCAL      | Só na função atual                    | Sempre que possível (padrão) |
| PRIVATE    | Função atual e funções filhas         | Passar dados implicitamente  |
| PUBLIC     | Todo o programa                       | Raramente — evitar           |
| STATIC     | Só na função, persiste entre chamadas | Contadores, flags de módulo  |

 **Regra de ouro:** na dúvida, use **LOCAL**. Você só sobe de escopo quando tiver um motivo claro.

## 3.4 Constantes

Constantes são valores que **não mudam** durante a execução do programa. Em Harbour/ADVPL usamos a diretiva **#define**:

```
#define PI          3.14159265
#define IVA         0.12
#define VERSAO      "2.0"
#define LIMITE_MAX  100

FUNCTION Main()
    LOCAL nRaio := 5
    LOCAL nArea

    nArea := PI * nRaio ^ 2
    QOut("Área: " + Str(nArea, 10, 2))

RETURN NIL
```

**Convenção:** nomes de constantes em **MAIÚSCULAS**.

## 3.5 Operadores

### Operadores aritméticos

| Operador   | Operação       | Exemplo | Resultado |
|------------|----------------|---------|-----------|
| +          | Adição         | 5 + 3   | 8         |
| -          | Subtração      | 10 - 4  | 6         |
| *          | Multiplicação  | 6 * 7   | 42        |
| /          | Divisão        | 15 / 4  | 3.75      |
| ^ ou **    | Potência       | 2 ^ 8   | 256       |
| % ou Mod() | Módulo (resto) | 10 % 3  | 1         |

### Precedência (ordem das operações)

Da maior para a menor precedência: 1. ^ (potência) 2. \*, /, % (multiplicação, divisão, módulo) 3. +, - (adição, subtração)

Use **parênteses** para alterar a ordem:

```
QOut(2 + 3 * 4)    // 14  (multiplicação primeiro)
QOut((2 + 3) * 4)  // 20  (parênteses primeiro)
```

## Operador de atribuição

```
nX := 10           // atribui 10 a nX
nX := nX + 5       // incrementa nX em 5 (resultado: 15)
nX++              // incrementa 1 (resultado: 16)
nX--              // decrementa 1 (resultado: 15)
```

## Operadores relacionais (comparação)

Retornam **.T.** ou **.F.** :

| Operador                                 | Significado    | Exemplo                                |
|------------------------------------------|----------------|----------------------------------------|
| <code>=</code> ou <code>==</code>        | Igual          | <code>5 == 5</code> → <b>.T.</b>       |
| <code>&lt;&gt;</code> ou <code>!=</code> | Diferente      | <code>5 &lt;&gt; 3</code> → <b>.T.</b> |
| <code>&lt;</code>                        | Menor que      | <code>3 &lt; 5</code> → <b>.T.</b>     |
| <code>&gt;</code>                        | Maior que      | <code>5 &gt; 3</code> → <b>.T.</b>     |
| <code>&lt;=</code>                       | Menor ou igual | <code>5 &lt;= 5</code> → <b>.T.</b>    |
| <code>&gt;=</code>                       | Maior ou igual | <code>6 &gt;= 5</code> → <b>.T.</b>    |

⚠ **Atenção (pega muita gente!):** em Harbour, `=` compara o **início** da string. Use `==` para comparação **exata**.

```
"Harbour" = "Harb"  // .T. (começa com "Harb")
"Harbour" == "Harb" // .F. (strings diferentes)
```

## Operadores lógicos

| Operador                                | Significado | Exemplo                                         |
|-----------------------------------------|-------------|-------------------------------------------------|
| <b>.AND.</b> ou <code>&amp;&amp;</code> | E lógico    | <b>.T.</b> <b>.AND.</b> <b>.F.</b> → <b>.F.</b> |
| <b>.OR.</b> ou <code>  </code>          | OU lógico   | <b>.T.</b> <b>.OR.</b> <b>.F.</b> → <b>.T.</b>  |
| <b>.NOT.</b> ou <code>!</code>          | NÃO lógico  | <b>.NOT.</b> <b>.T.</b> → <b>.F.</b>            |

Tabela verdade:



| A   | B   | A .AND. B | A .OR. B |
|-----|-----|-----------|----------|
| .T. | .T. | .T.       | .T.      |
| .T. | .F. | .F.       | .T.      |
| .F. | .T. | .F.       | .T.      |
| .F. | .F. | .F.       | .F.      |

Exemplo aplicado:

```

LOCAL nIdade := 25
LOCAL lAtivo := .T.

// Tem desconto se for maior de 60 E estiver ativo
IF nIdade >= 60 .AND. lAtivo
    QOut("Tem desconto")
ENDIF

// Acesso negado se menor de 18 OU não estiver ativo
IF nIdade < 18 .OR. .NOT. lAtivo
    QOut("Acesso negado")
ENDIF

```

## Operador de concatenação de strings

```

LOCAL cPrimeiro := "João"
LOCAL cSobrenome := "Silva"

QOut(cPrimeiro + " " + cSobrenome) // João Silva (mantém os espaços)
QOut(cPrimeiro - " " + cSobrenome) // JoãoSilva (o - remove espaços à direita do 1º)

```

## 3.6 Conversão de tipos

Harbour/ADVPL é **fracamente tipado** — o tipo de uma variável pode mudar. Mas é boa prática **converter explicitamente** quando necessário:

```

// Número → string
LOCAL nValor := 1500.50
QOut("Valor: " + Str(nValor))           // "Valor:      1500.50"
QOut("Valor: " + Str(nValor, 10, 2))    // "Valor:      1500.50"
QOut("Valor: " + Transform(nValor, "@E 999,999.99")) // formatado

// String → número
LOCAL cNumero := "42"
LOCAL nNumero := Val(cNumero)           // nNumero = 42

// Data → string
LOCAL dHoje := Date()
QOut(DToC(dHoje))                       // "26/03/2024"
QOut(DToS(dHoje))                       // "20240326" (formato YYYYMMDD)

// String → data
LOCAL dNasc := CtoD("15/03/1990")

// Lógico → string
QOut(If(.T., "Sim", "Não"))              // "Sim"

```

💡 A função `Str(nNumero, tamanho, decimais)` é sua melhor amiga para alinhar valores em relatórios de terminal.

## 3.7 Entrada de dados

Para ler dados do usuário no **terminal** (Harbour puro):

```

FUNCTION Main()
  LOCAL cNome
  LOCAL nIdade

  // ACCEPT lê uma string
  ACCEPT "Digite seu nome: " TO cNome

  // INPUT lê um valor (número, string, data, lógico)
  INPUT "Digite sua idade: " TO nIdade

  QOut("Olá, " + cNome + "!")
  QOut("Você tem " + Str(nIdade) + " anos.")

  RETURN NIL

```

**Nota:** no ambiente Protheus, a entrada de dados é feita por **formulários gráficos** ( `GET / READ` ). O `ACCEPT / INPUT` é usado apenas no terminal (Harbour puro), que é o que usamos nas aulas de fundamentos.

💡 **Truque seguro:** um padrão comum é ler tudo como texto com `ACCEPT` e depois converter com `Val()` . Assim você tem controle total do que entrou.

## 3.8 Exemplos práticos

### Exemplo 1 — Calculadora simples

```
FUNCTION Main()
  LOCAL nA, nB

  ACCEPT "Valor A: " TO nA
  nA := Val(nA)
  ACCEPT "Valor B: " TO nB
  nB := Val(nB)

  QOut("")
  QOut("=== Resultados ===")
  QOut("Soma:          " + Str(nA + nB, 10, 2))
  QOut("Subtração:      " + Str(nA - nB, 10, 2))
  QOut("Multiplicação: " + Str(nA * nB, 10, 2))

  IF nB <> 0
    QOut("Divisão:      " + Str(nA / nB, 10, 2))
  ELSE
    QOut("Divisão:      Impossível (divisão por zero)")
  ENDIF

  RETURN NIL
```

## Exemplo 2 — Área do triângulo

```
FUNCTION Main()
    LOCAL nBase, nAltura, nArea

    ACCEPT "Base do triângulo: " TO nBase
    nBase := Val(nBase)
    ACCEPT "Altura do triângulo: " TO nAltura
    nAltura := Val(nAltura)

    nArea := (nBase * nAltura) / 2

    QOut("")
    QOut("Área = " + Str(nArea, 10, 2) + " cm²")

    RETURN NIL
```

## Exemplo 3 — Desconto por idade

```
FUNCTION Main()
    LOCAL cNome, cNascStr, dNasc, nIdade, nPreco, nDesconto, nTotal

    ACCEPT "Nome: " TO cNome
    ACCEPT "Data de nascimento (DD/MM/AAAA): " TO cNascStr
    dNasc := CtoD(cNascStr)
    nIdade := (Date() - dNasc) / 365

    ACCEPT "Preço do produto: " TO nPreco
    nPreco := Val(nPreco)

    IF nIdade >= 60
        nDesconto := nPreco * 0.125 // 12,5% de desconto
        QOut("Desconto especial (60+): R$ " + Str(nDesconto, 8, 2))
    ELSE
        nDesconto := 0
    ENDIF

    nTotal := nPreco - nDesconto

    QOut("=== Resumo ===")
    QOut("Cliente : " + cNome)
    QOut("Idade   : " + Str(Int(nIdade)) + " anos")
    QOut("Preço    : R$ " + Str(nPreco, 8, 2))
    QOut("Desconto: R$ " + Str(nDesconto, 8, 2))
    QOut("Total    : R$ " + Str(nTotal, 8, 2))

    RETURN NIL
```



## Mão na massa (em aula)

Vamos construir **juntos**, ao vivo, a **calculadora simples** (Exemplo 1). Passo a passo:

1. Criar o arquivo `calculadora.prg`
2. Declarar as variáveis `LOCAL nA, nB`
3. Ler os dois valores com `ACCEPT` e converter com `Val()`
4. Exibir soma, subtração e multiplicação com `Str(..., 10, 2)`
5. Proteger a divisão com um `IF nB <> 0` (para não dividir por zero!)
6. Compilar e rodar:

```
hbm2 calculadora.prg
calculadora.exe
```

No fim, cada um testa a sua calculadora com valores diferentes. 🎯

---

## Referência rápida do Módulo 3

```
// TIPOS + PREFIXOS
LOCAL cTexto := "abc"      // c = caractere
LOCAL nNum    := 10        // n = numérico
LOCAL lFlag   := .T.       // l = lógico (.T. / .F.)
LOCAL dData   := Date()    // d = data
LOCAL aLista  := {}        // a = array

// ESCOPOS
LOCAL        // só na função (padrão)
PRIVATE      // função atual + filhas
PUBLIC       // global (evitar)
STATIC       // persiste entre chamadas

// OPERADORES
+ - * / ^ %      // aritméticos
:= ++ --         // atribuição / incremento
== <> < > <= >=  // relacionais (use == em strings!)
.AND. .OR. .NOT. // lógicos
"a" + "b"        // concatenar

// CONVERSÃO
Str(nNum, 10, 2) // número → texto (10 largura, 2 decimais)
Val("42")        // texto → número
DToC(dData)      // data → texto
CtoD("15/03/1990") // texto → data

// ENTRADA (terminal)
ACCEPT "Nome: " TO cNome // lê texto
INPUT  "Idade: " TO nIdade // lê valor

// CONSTANTE
#define PI 3.14159
```

## Exercícios

Os exercícios desta aula estão em **exercicios.md** (nesta mesma pasta). **Prazo sugerido:** até a Aula 4 (16/07).

## Para a próxima aula

**Módulo 4 — Decisões (condicionais).** Até agora seus programas executam **sempre** as mesmas instruções, de cima a baixo. Na próxima aula você vai ensinar o programa a **escolher**

**caminhos diferentes** dependendo dos dados, com `IF/ELSE` e `DO CASE` .

📖 Antes da Aula 4: rode a calculadora da aula e tente os exercícios de cálculo (círculo, hipotenusa, IMC). Travou? Poste no Discord [#duvidas-parte1-harbour](#) — a gente destrava junto.